

RI204

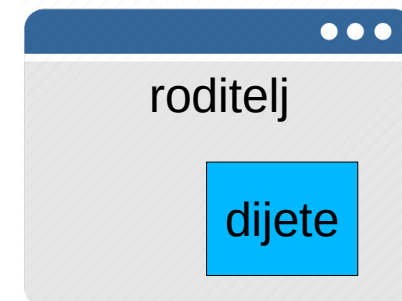
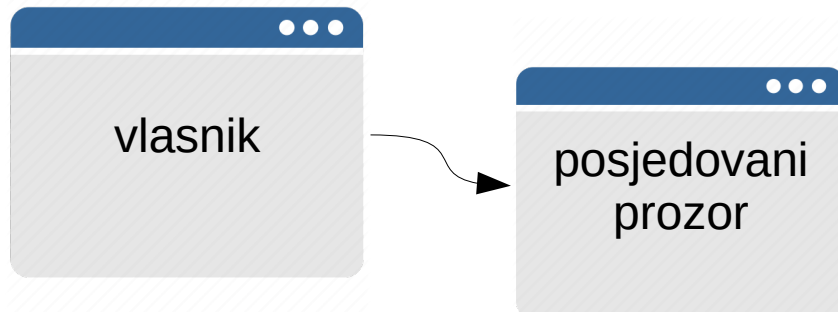
Windows programiranje

dr.sci. Edin Pjanić

- Dijete-prozor (child window)
 - Kreiranje dijete-prozora
 - Slanje poruka
 - Neke standardne kontrole
 - button
 - static
 - edit
 - listbox

Dijete prozor – child window

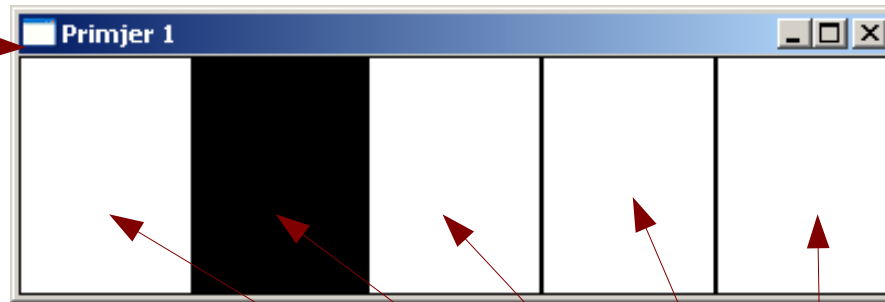
- Prozori najvišeg nivoa (top-level prozori)
 - su prozori koji se postavljaju direktno u desktop prostor. Svi do sada kreirani prozori su bili ovog tipa.
- Takav prozor se može postaviti u vezu sa drugim prozorima na način:
 - da je vlasnik drugog prozora = posjeduje drugi prozor
 - Posjedovani prozor se uništava kada se uništava vlasnik i sakriva kada se vlasnik minimizira. Prilikom kreiranja posjedovanog prozora istom se proslijeđuje handle od vlasnika (`hWndParent`)
 - da je roditelj drugog prozora = posjeduje dijete-prozor (child window)
 - Dijete prozor je ograničen unutar klijent područja roditelja. Prilikom kreiranja dijete-prozora potrebno je proslijediti handle od roditelja (`hWndParent`) i postaviti stil prozora na **WS_CHILD**.



Primjer 1

- Opis
 - Program dijeli klijent područje prozora na N jednakih pravougaonika. Svaki pravougaonik može se “upaliti” ili “ugasiti” lijevim klikom na njegovu površinu. Za $N = 5$ prozor ima izgled:

Top level prozor
klase “Roditelj”
sa window
procedurom
WinRoditelj



Child prozori klase
“Dijete”, sa win.
procedurom
WinDijete

- Dodaci u odnosu na primjer sa predavanja 1
 - Kreiraju se dvije klase “Roditelj” i “Dijete”

```
WNDCLASS      wc ;

wc.style      = CS_HREDRAW | CS_VREDRAW ;
wc.lpfnWndProc = WndRoditelj;
wc.cbClsExtra  = 0 ;
wc.cbWndExtra  = 0 ;
wc.hInstance   = hInstance ;
wc.hIcon       = LoadIcon (NULL, IDI_APPLICATION) ;
wc.hCursor     = LoadCursor (NULL, IDC_ARROW) ;
wc.hbrBackground = (HBRUSH) GetStockObject (WHITE_BRUSH) ;
wc.lpszMenuName = NULL ;
wc.lpszClassName = "Roditelj";
RegisterClass(&wc);
```

```
wc.lpszClassName = "Dijete";
wc.lpfnWndProc = WndDijete;
wc.hIcon = NULL;
wc.cbWndExtra = sizeof(long);
RegisterClass(&wc);
```

Registrujemo novu klasu. Svaki kreirani prozor klase “Dijete” dobija prostor za smještanje varijable (4 bajta). Ovu varijablu ćemo koristiti za čuvanje podatka o stanju prozora (on/off). Vrijednost varijable se može postaviti sa SetWindowLong a pročitati sa GetWindowLong

Win. procedura klase “Roditelj”

6

```
LRESULT CALLBACK WndRoditelj(HWND hwnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    static HWND djeca[N];
    int i, cx, cy;
    switch (message)
    {
        case WM_CREATE:
            for (i = 0; i < N; i++)
                djeca[i] = CreateWindow( "Dijete", NULL, WS_CHILD | WS_VISIBLE, 0, 0, 0, 0,
                    hwnd, NULL, (HINSTANCE)GetWindowLong(hwnd, GWL_HINSTANCE), NULL );
            return 0;

        case WM_DESTROY:
            PostQuitMessage (0) ;
            return 0 ;

        . . .
    }

    return DefWindowProc (hwnd, message, wParam, lParam) ;
}
```

Top-level prozor kreira i pamti niz handlea kreirane djece

child-prozor

Sredićemo poslije

Vraća handle na instancu programa (aplikacije) vlasnika prozora hwnd

Parent prozor

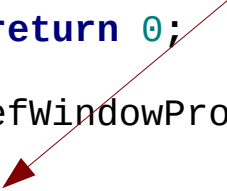
Win. procedura klase “Roditelj”

7

```
LRESULT CALLBACK WndRoditelj(HWND hwnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    static HWND djeca[N];
    int i,cx,cy;
    switch (message)
    {
        case WM_CREATE:
            for (i = 0; i < N; i++)
                djeca[i] = CreateWindow( "Dijete",NULL, WS_CHILD | WS_VISIBLE,0,0,0,0,
                    hwnd,NULL,(HINSTANCE)GetWindowLong(hwnd,GWL_HINSTANCE),NULL);
            return 0;

        case WM_DESTROY:
            PostQuitMessage (0) ;
            return 0 ;

        case WM_SIZE:
            cx = LOWORD(lParam) / N;
            cy = HIWORD(lParam);
            for ( i = 0; i < N; i++)
                MoveWindow(djeca[i],i*cx,0,cx,cy,TRUE);
            return 0;
    }
    return DefWindowProc (hwnd, message, wParam, lParam) ;
}
```



MoveWindow(hwnd, x, y, w, h, repaint); postavlja prozor hwnd na lokaciju x, y i mijenja širinu i visinu prozora w, h.

repaint određuje da li se tom prozoru šalje poruka za iscrtavanje prozora (WM_PAINT).

Win. procedura klase “Dijete”

```
LRESULT CALLBACK WndDijete(HWND hwnd, UINT message, WPARAM wParam, LPARAM lParam)
{
```

```
    HDC hdc;
    PAINTSTRUCT ps;
    RECT rect;
    switch (message)
    {
```

```
        case WM_CREATE:
```

```
            SetWindowLong(hwnd, 0, 0);
```

```
            return 0;
```

```
        case WM_LBUTTONDOWN:
```

```
            SetWindowLong(hwnd, 0, !GetWindowLong(hwnd, 0));
```

```
            InvalidateRect(hwnd, NULL, FALSE);
```

```
            return 0;
```

```
        case WM_PAINT:
```

```
            hdc = BeginPaint(hwnd, &ps);
```

```
            GetClientRect(hwnd, &rect);
```

```
            Rectangle(hdc, 0, 0, rect.right, rect.bottom);
```

```
            if (GetWindowLong(hwnd, 0) != 0)
```

```
                FillRect(hdc, &rect, GetStockObject(BLACK_BRUSH));
```

```
            EndPaint(hwnd, &ps);
```

```
            return 0;
```

```
    }
```

```
    return DefWindowProc(hwnd, message, wParam, lParam);
```

```
}
```

Postavlja vrijednost varijable dodijeljene prozoru hwnd na 0 (treći parametar).

Drugi parametar je ofset pozicija u **bajtima** gdje se upisuje vrijednost varijable. Ukupan broj bajta koji je na raspolaganju korisniku definisan je pri kreiranju klase poljem cbWndExtra.

Čita vrijednost 32-bitne cjelobrojne varijable dodijeljene prozoru hwnd. Drugi parametar je ofset u **bajtima** odakle se varijabla čita.

- Prozori međusobno komuniciraju putem poruka
 - Postoje dva načina slanja poruka:



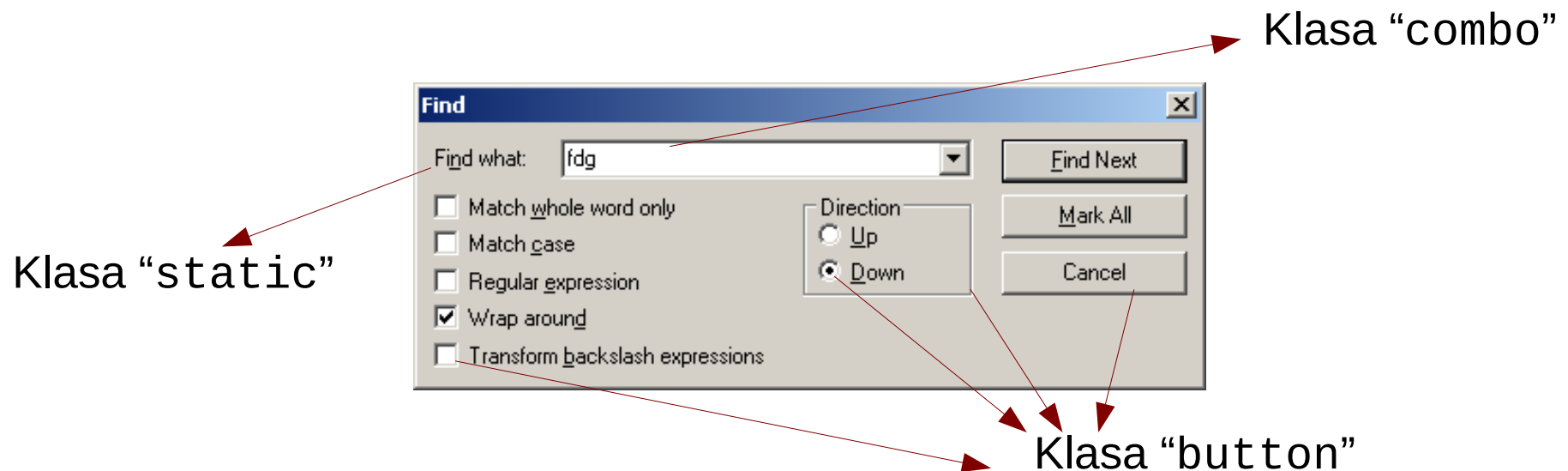
```
LRESULT SendMessage(HWND hwnd,  
                    UINT message,  
                    WPARAM wParam,  
                    LPARAM lParam);
```

```
BOOL PostMessage(HWND hwnd,  
                UINT message,  
                WPARAM wParam,  
                LPARAM lParam);
```



- **SendMessage radi sinhrono:**
 - šalje poruku message prozoru hwnd sa parametrima wParam i lParam, te čeka da se ta poruka tretira a rezultat je tipa LRESULT vrijednosti koju vrati win procedura prozora hwnd
 - garantuje tretiranje poruke do trenutka kada SendMessage vrati rezultat
- **PostMessage radi asinhrono:**
 - poruku prosljeđuje u red poruka destinacijskog prozora hwnd i ne garantuje da će ta poruka biti tretirana.
- Poruka WM_USER rezervisana je za korisničke poruke za koje wParam i lParam mogu imati proizvoljne vrijednosti

- Opis
 - Kontrole su **djeca-prozori** koji pripadaju klasama koje su skupa sa svojim wnd. procedurama definisane unutar dll-ova koji dolaze sa operativnim sistemom.
 - Korištenjem kontrola postiže se konzistentan izgled i ponašanje GUI aplikacija.
 - Neke GUI biblioteke same iscrtavaju svoje kontrole i definišu njihovo ponašanje, npr. FLTK, GTK itd.
 - Na slici je dat izgled nekih standardnih windows kontrola skupa sa imenima klasa kojima pripadaju.



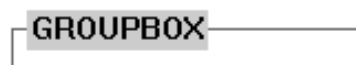
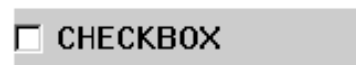
▪ Kreiranje

- Kontrole se kreiraju identično kao bilo koji dijete-prozor, pomoću funkcije `CreateWindow`. Nije potrebno registrovati klasu kontrole jer ista već postoji unutar OS-a sa predefinisanim imenom.
- Parametar `hMenu` prilikom kreiranja kontrole interpretira se kao identifikacioni broj (ID) kontrole (kontrole nemaju menija) i postavlja se na neku unikatnu vrijednost, obično makroom, kao npr:
 - `#define DUGME_1 101`
 - Ukoliko je poznat handle kontrole `hwnd` njen ID se može dobiti:
 - `id = GetWindowLong(hwnd, GWL_ID);` ili
 - `id = GetDlgCtrlID(hwnd);`
 - Obrnuto, ukoliko je poznat ID kontrole `id` i handle roditelja `hwndR`, handle kontrole `hwnd` se može dobiti:
 - `hwnd = GetDlgItem(hwndR, id);`
- Kontrole se obično slažu u dijalog-prozore.

- **Komunikacija**

- Komunikacija se najčešće obavlja pomoću SendMessage funkcije
 - Ovisno o klasi i/ili stila prozora kontrole, kontrola prima različite tipove poruka.
- Kontrola obično obavještava roditelja o promjeni stanja porukom WM_COMMAND, gdje su:
 - lParam - handle kontrole
 - LOWORD(wParam) - ID kontrole npr DUGME_1
 - HIWORD(wParam) - notifikacioni kod poruke, npr. za kliknuto dugme (kontrola button) notifikacioni kod je BN_CLICKED

- klasa “button”
 - predstavlja se u obliku malog pravougaonog prozora sa ili bez teksta na kojeg korisnik može kliknuti kako bi promijenio njegovo stanje.
 - prilikom interakcije sa korisnikom trajno ili privremeno mijenja izgled na ekranu
 - u zavisnosti od stila, pojavljuje se sa različitim izgledom.



- Bez obzira na stil, klik na dugme generiše roditelju WM_COMMAND poruku sa HIWORD(wParam) vrijednosti BN_CLICKED.
- Postavljanje ili provjera stanja za radio ili check kontrolu h_dugme vrši se porukama:
 - Provjera:
 - `st = (int) SendMessage(h_dugme, BM_GETCHECK, 0, 0)`
 - Postavljanje na uključeno stanje:
 - `SendMessage(h_dug, BM_SETCHECK, BST_CHECKED, 0);`

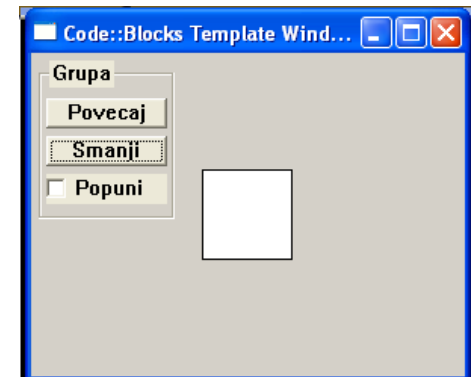
Primjer 2

14

```
LRESULT CALLBACK WndProc (HWND hwnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    RECT rect; HBRUSH hbr; PAINTSTRUCT ps; HDC hdc;
    static int stranica = 40;
    int x_s, y_s;
    switch (message)
    {
        case WM_CREATE:
        {
            HINSTANCE hinst = (HINSTANCE) GetWindowLong(hwnd, GWL_HINSTANCE);
            CreateWindow("button", "Grupa", WS_CHILD | WS_VISIBLE | BS_GROUPBOX,
                5, 5, 90, 105, hwnd, NULL, hinst, NULL);
            CreateWindow("button", "Povecaj", WS_CHILD | WS_VISIBLE | BS_PUSHBUTTON,
                10, 30, 80, 20, hwnd, (HMENU) IDB_DUGME1, hinst, NULL);
            CreateWindow("button", "Smanji", WS_CHILD | WS_VISIBLE | BS_PUSHBUTTON,
                10, 55, 80, 20, hwnd, (HMENU) IDB_DUGME2, hinst, NULL);
            CreateWindow("button", "Popuni", WS_CHILD | WS_VISIBLE | BS_CHECKBOX,
                10, 80, 80, 20, hwnd, (HMENU) IDB_CHECK1, hinst, NULL);
            return 0;
        }
        case WM_COMMAND:
        {
            switch (LOWORD(wParam))
            {
                case IDB_DUGME1:
                    stranica += 10;
                    break;
                case IDB_DUGME2:
                    stranica -= 10;
                    break;
                case IDB_CHECK1:
                    SendMessage((HWND) lParam, BM_SETCHECK,
                        !SendMessage((HWND) lParam, BM_GETCHECK, 0, 0), 0);
                    break;
            }
            InvalidateRect(hwnd, NULL, TRUE);
            return 0;
        }
    }
}
```

```
#define IDB_DUGME1 101
#define IDB_DUGME2 102
#define IDB_CHECK1 103
```

e



```
case WM_PAINT:
    hdc = BeginPaint(hwnd,&ps);
    hbr = SendMessage(GetDlgItem(hwnd,IDB_CHECK1),BM_GETCHECK,0,0) ?
        GetStockObject(BLACK_BRUSH) : GetStockObject(WHITE_BRUSH);
    SelectObject(hdc,hbr);
    GetClientRect(hwnd,&rect);
    x_s = (rect.right - stranica) / 2; y_s = (rect.bottom - stranica) / 2;
    Rectangle(hdc,x_s,y_s,x_s + stranica, y_s + stranica);
    EndPaint(hwnd,&ps);
    return 0;
case WM_DESTROY:
    PostQuitMessage (0) ;
    return 0 ;
}
return DefWindowProc (hwnd, message, wParam, lParam) ;
}
```

- klasa “static”
 - Jednostavna kontrola koja obično unutar pravougaonika prikazuje tekst. Najčešće služi kao oznaka neke druge kontrole.
- klasa “edit”
 - je dijete-prozor unutar kojeg korisnik može vršiti unos teksta pomoću tastature. Klikom na edit kontrolu, kontrola dobija fokus te prikazuje vertikalnu liniju (caret) na lokaciji na kojoj je moguć unos teksta.
 - Tekst je moguće selektirati, kopirati i brisati. Caret je moguće pomjerati pomoću tastature ili miša.

- Neki stilovi
 - Tekst pozicija lijevo, desno i centriran `ES_LEFT`, `ES_RIGHT` i `ES_CENTER`.
 - `WS_HSCROLL` i `WS_VSCROLL` dodaje kontrole za skrolanje edit kontrole
 - `ES_MULTILINE` dodaje mogućnost unosa teksta u više linija
 - `ES_AUTOHSCROLL` i `ES_AUTOVSCROLL` aktivira automatsko skrolanje kontrole
- Neke notifikacione poruke od edit kontrole
 - `EN_CHANGE` sadržaj je izmijenjen
 - `EN_SELCHANGE` došlo je do izmjene selekcije teksta
 - `EN_MAXTEXT` dostignut maksimalno dozvoljen broj karaktera
- Neke poruke za edit kontrolu
 - `br_l = SendMessage(hwndE, EM_GETLINECOUNT, 0, 0);` vraća broj linija u kontroli
 - `br_c = SendMessage(hwndE, EM_LINELENGTH, br_l, 0);` vraća broj karaktera u liniji broj `br_l`
 - `SendMessage(hwndE, EM_GETLINE, br_l, (LPARAM) szBuffer);` kopira liniju pod brojem `br_l` u string `szBuffer`

Primjer 3 – Suma dva broja

18

```
LRESULT CALLBACK WndProc (HWND hwnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    static int br1 = 0, br2 = 0;
    switch (message)
    {
        case WM_CREATE:
        {
            HINSTANCE hinst = (HINSTANCE) GetWindowLong(hwnd, GWL_HINSTANCE);
            CreateWindow("edit", "broj 1", WS_CHILD | WS_VISIBLE | WS_BORDER | ES_LEFT,
                10, 30, 80, 20, hwnd, (HMENU) 101, hinst, NULL);
            CreateWindow("edit", "broj 2", WS_CHILD | WS_VISIBLE | WS_BORDER | ES_LEFT,
                10, 55, 80, 20, hwnd, (HMENU) 102, hinst, NULL);
            CreateWindow("static", "", WS_CHILD | WS_VISIBLE | SS_CENTER,
                100, 30, 120, 20, hwnd, (HMENU) 103, hinst, NULL);
            return 0;
        }
        case WM_COMMAND:
        {
            char buffer[20];
            switch(LOWORD(wParam))
            {
                case 101:
                {
                    if(HIWORD(wParam) == EN_CHANGE)
                    {
                        GetWindowText((HWND) lParam, buffer, 20);
                        br1 = atoi(buffer);
                    }
                    break;
                }
                case 102:
                {
                    if(HIWORD(wParam) == EN_CHANGE)
                    {
                        GetWindowText((HWND) lParam, buffer, 20);
                        br2 = atoi(buffer);
                    }
                    break;
                }
            }
        }
    }
}
```

```
        if ( (br1 != 0) && (br2 != 0))
        {
            sprintf(buffer, "Suma je: %5d", br1 + br2);
            SetWindowText(GetDlgItem(hwnd, 103), buffer);
        }
        else
            SetWindowText(GetDlgItem(hwnd, 103), "");
        return 0;
    }
    case WM_DESTROY:
        PostQuitMessage (0) ;
        return 0 ;
}
return DefWindowProc (hwnd, message, wParam, lParam) ;
}
```

- Klasa “list”
 - je dijete-prozor koji se sastoji od liste stringova koja se da skrolati.
 - program može dodavati ili oduzimati stringove iz liste slanjem poruka kontroli
 - kontrola putem WM_COMMAND poruke obavještava prozor-roditelj o selekciji (odabiru) stringa iz liste.
 - Da bi se generisale WM_COMMAND poruke potrebno je specificirati stil LBS_NOTIFY
 - selekciju elementa u listi moguće je izvršiti putem miša ili tastature
 - Kada kontrola ima fokus moguće je pomjeranje pomoću strelica i učiniti izbor sa SPACE
 - Moguće je specificirati listu sa višestrukim izborom
 - Stil LBS_MULTIPLESEL
 - Stil LBS_SORT generiše sortiranu listu

- `SendMessage(hwndList, LB_ADDSTRING, 0, (LPARAM) szString);`
Dodaje string `szString`
- `SendMessage(hwndList, LB_INSERTSTRING, iIndex, (LPARAM) szString);`
 - Insertuje string `szString` na poziciju `iIndex`
- `SendMessage (hwndList, LB_DELETESTRING, iIndex, 0) ;`
 - Briše string sa pozicije `iIndex`
- `SendMessage (hwndList, LB_RESETCONTENT, 0, 0) ;`
 - Resetuje listu
- `iCount = SendMessage (hwndList, LB_GETCOUNT, 0, 0) ;`
 - Vraća broj stringova u listi
- `iIndex = SendMessage (hwndList, LB_GETCURSEL, 0, 0) ;`
 - Vraća indeks trenutno selektovanog stringa
- `iLength = SendMessage (hwndList, LB_GETTEXTLEN, iIndex, 0) ;`
 - Vraća dužinu stringa sa indeksom `iIndex`
- `iLength = SendMessage (hwndList, LB_GETTEXT, iIndex, (LPARAM) szBuffer) ;`
 - Puni `szBuffer` stringom na pozicij `iLength`

- Sljedeći notifikacioni kodovi dolaze sa WM_COMMAND porukom:
 - LBN_SELCHANGE promjena selekcija
 - LBN_DBLCLK dvostruki klik na neki string
 - LBN_ERRSPACE lista ne može alocirati potrebnu memoriju
 - LBN_KILLFOCUS lista je izgubila trenutni fokus
 - LBN_SETFOCUS lista je primila fokus

```
LRESULT CALLBACK WndProc (HWND hwnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    char str1[20];
    HWND ListBox = NULL;
    switch (message)
    {
        case WM_CREATE:
            ListBox = CreateWindow ("listbox",NULL,WS_CHILD | WS_VISIBLE | LBS_NOTIFY |
                                   WS_VSCROLL | WS_BORDER,
                                   10,10,50,100,hwnd,(HMENU) 101,(HINSTANCE)GetWindowLong(hwnd,GWL_HINSTANCE), NULL);
            SendMessage(ListBox,LB_ADDSTRING,0,(LPARAM)"jedan");
            SendMessage(ListBox,LB_ADDSTRING,0,(LPARAM)"dva");
            SendMessage(ListBox,LB_INSERTSTRING,1,(LPARAM)"tri");
            return 0;
        case WM_COMMAND:
            if ((LOWORD (wParam) == 101) && (HIWORD (wParam) == LBN_SELCHANGE))
            {
                int iIndex = SendMessage (GetDlgItem(hwnd,101),LB_GETCURSEL,0,0);
                SendMessage (GetDlgItem(hwnd,101),LB_GETTEXT,iIndex,(LPARAM)str1);
                MessageBox (hwnd, str1, "Odabrali ste", MB_OK);
            }
            return 0;
        case WM_DESTROY:
            PostQuitMessage (0) ;
            return 0 ;
    }
    return DefWindowProc (hwnd, message, wParam, lParam) ;
}
```